

Multithreaded Cricket Simulator

Samarth Maheshwari (24114084)
Vishal Kumar Shaw (24114105)

1 Introduction

This project simulates a T20 cricket match between **India and New Zealand** using Operating System concepts such as:

- **Multithreading:** Multiple players act simultaneously, threads are used to model batsman, bowler, and fielders.
- **Synchronization:** Shared resources like pitch and crease cause race conditions, so mutex locks, semaphores, and condition variables are used to ensure safe access.
- **CPU Scheduling:** Multiple batsmen waiting to play and bowlers waiting to bowl creates scheduling issues, so SJF, FCFS, Priority and Round Robin are used for execution order.
- **Deadlock in Running:** Circular wait between batsmen during runs causes deadlock, so it is resolved using forced run-out.
- **Run-out by Fielder:** Batsman and fielders compete for crease, so whoever acquires the critical section first wins.
- **Toss Decision:** Match start requires choosing batting/bowling, so random selection simulates the toss.
- **Game Uncertainty:** We use probability to determine runs, wickets, wides, and no-balls.

Each player is modeled as a thread, and match execution resembles process scheduling in an Operating System.

2 System Design

2.1 Concept Mapping

Cricket Concept	OS Concept
Batsman Threads	Consumer Processes
Bowler Thread	Producer Process
Fielders	Competing Threads (Race Participants)
Ball Delivery	Task / Job Execution
Over	Time Slice (CPU Scheduling Unit)
Scheduler (SJF / FCFS / RR / Priority)	CPU Scheduling Algorithms
Running Between Wickets	Resource Acquisition by Threads
Run-out (Fielder wins)	Race Condition Outcome
Deadlock (Both Batsmen Stuck)	Circular Wait Deadlock
Deadlock Resolution (Run-out)	Deadlock Recovery Mechanism
Scoreboard	Shared Data Structure
Match State Variables	Shared Memory

2.2 Producer-Consumer Model

In this simulation, the interaction between the bowler and batsman is modeled using **Producer-Consumer problem** from Operating Systems.

- **Batsman as Consumer:** Batsman consumes the ball produced and determines outcome (runs/wicket).
- **Bowler as Producer:** Bowler produces ball events and places them on the pitch.
- **Role of Fielder:** Run attempts create competition for crease, so fielders act as competing threads trying to acquire crease for run-out.
- **Over as Time Slice:** Continuous execution needs fairness, so each over acts as a time slice enabling Round Robin and Priority(Death overs) scheduling of bowlers.
- **Crease as Critical Section:** Multiple players accessing crease causes conflict, so it is protected as a critical section using locks.
- **Pitch as Shared Resource:** Ball state is shared between bowler and batsman, hence pitch acts as a shared variable which is synchronized by using mutex.

This model ensures proper coordination between threads and prevents race conditions thus matching the producer-consumer synchronization problem.

3 Data Structures

```

1 // Represents a single batsman in the match
2 typedef struct {
3     int id;                // Unique identifier for each batsman
4     char name[50];         // Player name
5     char role[20];         // Role (Batsman, All-Rounder, etc.)
6
7     int runs;              // Total runs scored
8     int balls_faced;       // Number of balls faced
9     int fours;             // Number of 4s
10    int sixes;              // Number of 6s
11
12    int is_out;             // 1 if dismissed, 0 otherwise
13
14    int job_length;         // Used for scheduling (lower = higher
15                            // priority in SJF)
16    int in_crease;          // Indicates if batsman is currently on
17                            // field (resource occupancy)
18
19    int turn_around_time    // Time from arrival to dismissal (used
20                            // for analysis)
21 } Batsman;
22
23 // Represents a bowler thread (producer of ball events)
24 typedef struct {
25     int id;                // Unique identifier
26     char name[50];         // Bowler name
27
28     int balls_bowled;       // Total balls delivered
29     int runs_given;        // Runs conceded
30     int wickets;           // Wickets taken
31
32     int skill;             // Skill level affecting outcome
33                            // probabilities
34 } Bowler;
35
36 // Represents the current score of the match
37 typedef struct {
38     int runs;              // Total runs scored
39     int wickets;           // Total wickets fallen
40     int overs;             // Completed overs
41     int balls;             // Balls in current over

```

```

40     int extras;                // Extra runs (wides, no-balls)
41 } Score;
42
43
44 // Maintains global state of the match shared across threads
45 typedef struct {
46     Score score;                // Current score
47
48     int striker;                // Index of striker
49     int non_striker;            // Index of non-striker
50     int next_batsman;            // Next batsman to come
51
52     int ball_in_air;            // Indicates if ball is active (used
                                // for fielders)
53     int match_running;          // Controls thread execution
                                // lifecycle
54     int run_in_progress;        // Indicates if batsmen are
                                // attempting a run
55 } MatchState;
56
57
58 // Represents a team (collection of players and bowlers)
59 typedef struct {
60     Batsman players[11];        // Batting lineup (also used as
                                // fielders)
61     Bowler bowlers[6];          // Available bowlers
62     char name[50];              // Team name
63 } Team;

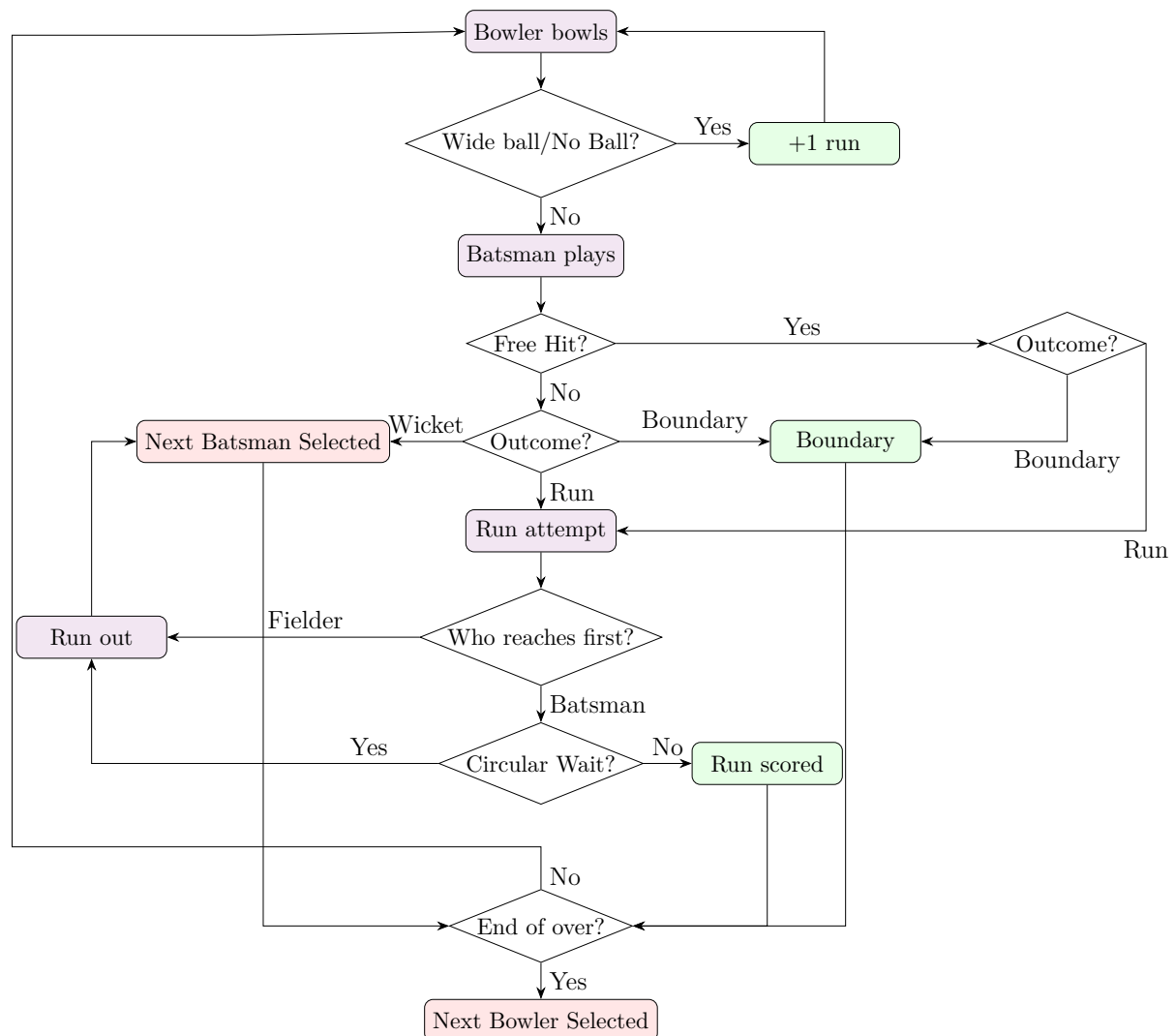
```

4 Thread Model

4.1 Types of Threads

- Bowler Threads (6)
- Batsman Threads (2 active, total 11)
- Fielder Threads(including Bowlers) (11)

4.2 Execution Flow Diagram



5 Synchronization

All threads are running at the same time. All of them share common data like the score, the pitch state, and the crease. If two threads try to read and write the same variable at the same time without any coordination, the data gets corrupted (e.g. if two threads both add runs to the score simultaneously, one update can overwrite the other (i.e. will cause **race condition**)).

5.1 Mutex Locks

A **mutex** (mutual exclusion lock) solves this by making sure only one thread can enter a critical section at a time. Every other thread that tries to enter has to wait until the first one is done.

```

1 pthread_mutex_t pitch_mutex;
2 pthread_mutex_t score_mutex;
3 pthread_mutex_t crease_mutex;
4 pthread_mutex_t batsman_mutex;

```

```

5 pthread_mutex_t crease_state_mutex;
6 pthread_mutex_t run_mutex;
7 pthread_mutex_t fielder_mutex;
8 pthread_mutex_t end1_mutex;
9 pthread_mutex_t end2_mutex;
10 pthread_mutex_t deadlock_mutex;
11 pthread_mutex_t print_mutex;
12 pthread_mutex_t start_mutex;

```

- **pitch_mutex**

`pitch_ball` is the shared buffer between the bowler and striker — the bowler writes 1 (ball delivered) and the striker resets it to -2 (consumed). Without this mutex the striker could start processing a ball the bowler hasn't finished writing or the bowler could deliver a new ball while the striker is still playing the previous one. *Involved: bowler and striker threads.*

- **score_mutex**

After every ball `update_score()` adds runs or increments wickets. This mutex ensures the write completes fully and atomically before any other thread reads the score, preventing partially updated values from being seen. *Involved: striker thread.*

- **crease_mutex**

This is the run-out race lock. When the ball is hit for 1, 2, or 3 runs, the striker and all active fielder threads simultaneously call `pthread_mutex_trylock()` on this mutex. Whoever grabs it first wins striker means safe run fielder means run-out. `trylock` is used (not blocking lock) so the loser doesn't hang, it just accepts the outcome and moves on. *Involved: striker and all 11 fielder threads.*

- **batsman_mutex**

All 11 batsman threads are alive throughout the match but only 2 are active at any time. The rest wait on `batsman_cond` inside this mutex, repeatedly checking if their ID matches `match.striker` or `match.non_striker`. The striker broadcasts on this condition after every wicket, every over-end, and at innings start to wake the relevant batsman. *Involved: all 11 batsman threads.*

- **crease_state_mutex**

Before calling `sem_wait()` to enter the crease, a batsman thread checks the `in_crease` flag to avoid waiting twice. But reading the flag and acting on it must be atomic otherwise two threads could both read `in_crease = 0` and both call `sem_wait()`, incorrectly consuming two semaphore slots. *Involved: striker and non-striker threads.*

- **run_mutex**

The non striker should not start running until the fielder phase is fully resolved. The striker sets `run_ready = 1` and signals the non striker through `run_cond` only after the fielder outcome comes. This mutex guards that flag so the signal and the flag update happen together cleanly. *Involved: striker and non-striker threads.*

- **fielder_mutex**

When a ball is hit all 11 fielder threads share variables like `ball_in_air`, and `fielder_runout_happened`, `winning_fielder_id`, and `active_fielders_count`. Both the striker (that is who sets up the race) and the fielders (that is who update the outcome) access these variables concurrently. This mutex ensures they don't overwrite each other's changes. *Involved: striker and all 11 fielder threads.*

- **end1_mutex and end2_mutex**

These represent the two physical ends of the pitch. The striker always locks `end1` first and then tries `end2`. The non-striker always locks `end2` first and tries `end1`. This ordering creates a situation where each holds one resource and waits for the other (i.e. **circular wait**) . *Involved: striker and non-striker threads.*

- **deadlock_mutex**

When no batsman(i.e both striker and non striker) can complete the run, both set their `_waiting` flags to 1. `detect_deadlock()` must read both flags as a consistent snapshot if it reads one flag and the other thread changes the second flag in between the detection gives a wrong answer. This mutex ensures both flags are always read together atomically. *Involved: striker and non-striker threads.*

- **print_mutex**

Multiple threads print to the terminal ball logs, over summaries, wicket messages, deadlock alerts. Without this lock, output from two threads printing at the same time gets mixed together into unreadable gibberish. Every `printf()` in the simulation is wrapped inside this mutex. *Involved: striker and main threads.*

- **start_mutex**

This guards the `innings_started` flag, which acts as a global on/off switch for all threads. At the start of each innings the main thread sets it to 1 and broadcasts; at the end it sets it to 0 and broadcasts again. All threads check this flag in a loop and sleep whenever it is 0, effectively pausing the entire simulation between innings. *Involved: all threads and the main thread.*

5.2 Semaphore

A **semaphore** is a synchronization tool that maintains a counter. Threads call `sem_wait()` to decrement the counter and proceed, or block if the counter is already 0. They call `sem_post()` to increment it and wake a waiting thread.

```
1 sem_t crease_sem;
```

`crease_sem` is initialized to 2 because exactly two batsmen are allowed on the field at any time. When a batsman enters the crease, they call `sem_wait()`, decrementing the count. When dismissed, they call `sem_post()`, freeing the slot for the incoming batsman. If a third thread somehow tried to enter, it would block until one dismissal frees a slot. This enforces the two-batsmen rule as a hard OS-level guarantee, not just a software check. *Involved: striker and non-striker threads.*

5.3 Condition Variables

A **condition variable** lets a thread sleep and wait for a specific event rather than spinning in a loop checking a flag repeatedly. The thread sleeps inside a mutex, and when another thread signals the condition, the sleeping thread wakes up, re-acquires the mutex, and checks its predicate again before continuing.

```
1 pthread_cond_t ball_hit_cond;  
2 pthread_cond_t run_cond;  
3 pthread_cond_t batsman_cond;  
4 pthread_cond_t start_cond;
```

- **ball_hit_cond** (paired with **fielder_mutex**)

All 11 fielder threads sleep on this condition between deliveries. When the striker hits the ball for 1, 2, or 3 runs, it sets **ball_in_air** = 1 and calls **broadcast()** to wake all fielders at once. Each fielder independently decides whether to chase (2% chance) and then races toward **crease_mutex**. Broadcast is used instead of signal because all fielders need to compete simultaneously. *Striker broadcasts; all 11 fielder threads wait.*

- **run_cond** (paired with **run_mutex**)

The non-striker sleeps here waiting for **run_ready** = 1. The striker only sets this flag after the fielder race is fully resolved — meaning it is definitively known whether a run-out happened or not. This ensures the non-striker's **attempt_run()** call always happens with a clear and final outcome already decided. *Striker signals; non-striker waits.*

- **batsman_cond** (paired with **batsman_mutex**)

Nine out of eleven batsman threads are sleeping on this condition at any given time. Whenever the lineup changes — a wicket, an over-end strike swap, or innings start — the striker calls **broadcast()**. Every batsman wakes up, checks if its ID now matches the striker or non-striker slot, and either takes over or goes back to sleep. The **while** loop around the wait handles spurious wakeups correctly. *Striker broadcasts; all 11 batsman threads wait.*

- **start_cond** (paired with **start_mutex**)

The main thread broadcasts on this condition at the start and end of every innings. All threads wait on it whenever **innings_started** = 0. This keeps every thread neatly parked while the match state is reset between innings, and resumes them all together when the next innings begins. *Main thread broadcasts; all threads wait.*

Note on lost wakeups: Every condition variable is paired with an integer flag (**run_ready**, **innings_started**, **ball_in_air**). The flag is always set *before* signalling, and always checked inside a **while** loop on the waiting side. This handles the case where the signal fires before the thread has called **wait()** — instead of sleeping forever, the thread sees the flag is already set and skips the wait entirely.

5.4 Run outs through Race conditions and Deadlocks

In this simulation, a run-out can happen in two different ways — one through a direct race between a fielder and a batsman, and another through a deadlock caused by a circular wait between the two batsmen. Both are explained below.

Type 1: Fielder Run-Out (Race on `crease_mutex`)

When the striker hits the ball for 1, 2, or 3 runs, a real-time race condition is triggered. The striker and the active fielder threads simultaneously attempt to acquire the `crease_mutex` using `pthread_mutex_trylock()`. This mutex models reaching the crease safely.

Fielders do not always participate; each fielder is activated probabilistically (2% independently). All active fielders then compete in the race.

- If the striker acquires `crease_mutex` first, the run is considered safe, and the non-striker is signalled to proceed.
- If a fielder acquires it first, `fielder_runout_happened` is set, and the batsman is declared out.

`pthread_mutex_trylock()` is used instead of a blocking lock to model a true race: threads that fail to acquire the mutex do not wait but immediately proceed based on the outcome, preventing unnecessary blocking.

The dismissed batsman is determined based on the number of runs completed before the run-out:

```
victim = (runs_completed % 2 == 0) ? striker : non_striker
```

Type 2: Deadlock Run-Out (Circular Wait on `end1_mutex` / `end2_mutex`)

Circular Deadlock Scenario: During a run attempt, both the striker and non-striker try to reach the opposite ends. Each batsman attempts to acquire locks on the two ends of the pitch (represented by `end1_mutex` and `end2_mutex`). A **circular wait** situation occurs when:

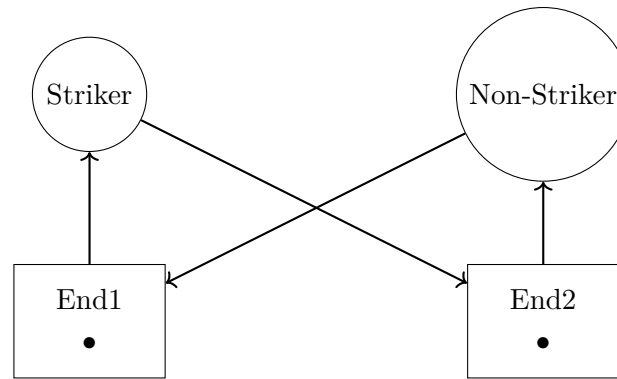
- Striker acquires one end (say `end1`) and waits for the other end (`end2`).
- Non-striker acquires the other end (`end2`) and waits for `end1`.

This creates a condition where both threads are waiting for each other indefinitely, resulting in a **deadlock**. This satisfies the circular wait condition of deadlock in Operating Systems.

Deadlock Detection: To detect this situation, flags such as *striker_waiting* and *non-striker_waiting* are used.

- If both flags are set simultaneously, it indicates that both batsmen are waiting.

- This confirms the presence of a circular deadlock.



Deadlock Resolution (Forced Run-out): Instead of allowing the system to hang, the deadlock is resolved using a forced run-out mechanism:

- One of the batsmen is declared out (run-out).
- The selection depends on the running state (striker or non-striker).
- Both locks are released safely to restore system execution.

```

1 // Striker side
2 pthread_mutex_lock(&end1_mutex);
3 if (pthread_mutex_trylock(&end2_mutex) != 0) {
4     striker_waiting = 1;
5     pthread_mutex_unlock(&end1_mutex);
6     return 1; // deadlock flag
7 }
8
9 // Non-striker side
10 pthread_mutex_lock(&end2_mutex);
11 if (pthread_mutex_trylock(&end1_mutex) != 0) {
12     nonstriker_waiting = 1;
13     pthread_mutex_unlock(&end2_mutex);
14     return 1;
15 }

```

Summary: Both run-out types use mutex races to determine the outcome — the fielder run-out is a speed race on `crease_mutex`, while the deadlock run-out is a resource contention problem on `end1_mutex` and `end2_mutex` that the simulator detects and resolves like an OS kernel handling a deadlock.

6 Scheduling Algorithms

This simulator uses four scheduling algorithms — **batsman selection** (who bats next) uses SJF or FCFS, and **bowler selection** (who bowls next) uses Round Robin or Priority

scheduling.

6.1 Shortest Job First (SJF) — Batsman Scheduling

In SJF, the batsman with the smallest `job_length` is picked from the ready queue first. Here, `job_length` represents the batsman's skill/expected workload, a lower value means a weaker batsman who is expected to face fewer balls. So SJF essentially sends the weakest batsmen to the crease first.

The ready queue is an array of batsman indices (`ready_queue[11]`). When `get_next_batsman()` is called and `scheduling_type == 0` (SJF mode), it searches the entire queue to find the index with the minimum `job_length`, removes it from the queue, and returns it.

```

1 int get_next_batsman() {
2     if (rq_size == 0) return -1;
3
4     int idx = 0;
5
6     if (scheduling_type == 0) // SJF: find minimum job_length in
       queue
7     {
8         for (int i = 1; i < rq_size; i++) {
9             if (batsmen[ready_queue[i]].job_length
10                batsmen[ready_queue[idx]].job_length)
11                 idx = i;
12         }
13     }
14     // remove chosen batsman from ready queue
15     int chosen = ready_queue[idx];
16     for (int i = idx; i < rq_size - 1; i++)
17         ready_queue[i] = ready_queue[i + 1];
18     rq_size--;
19
20     return chosen;
21 }

```

OS Mapping: Each batsman is a process with a known burst time (`job_length`). SJF picks the shortest burst time process from the ready queue first, which minimizes average waiting time. **Observation:** SJF gives lower average waiting time for the middle-order batsmen (positions 3–6) since weaker tailenders get sent in earlier so that it keeps stronger batsmen waiting longer.

6.2 First Come First Serve (FCFS) — Batsman Scheduling

In FCFS mode (`scheduling_type == 1`), `get_next_batsman()` uses the same function but skips the SJF search. Since `idx` stays at 0, it always picks the first batsman remaining in the ready queue.

```

1 // FCFS path: scheduling_type == 1
2 // idx stays 0, so ready_queue[0] is always picked
3 // batsmen come in as: 0, 1, 2, 3 ... 10 (batting order)
4 int chosen = ready_queue[0];

```

OS Mapping: Processes are scheduled in the order they arrive, there is no reordering based on burst time. It is simple and fair, but it can lead to higher waiting times if a long burst process (strong batsman) is picked early and occupies the CPU (crease) for a long time.

Observation: FCFS produces higher average wait times for the middle order compared to SJF, because strong top order batsmen (high `job_length`) come in first and face more balls, making others wait longer.

6.3 Round Robin — Bowler Scheduling

Round Robin is used to rotate bowlers after every over (overs 1 through 19). After each over ends, `round_robin_scheduler()` is called, which simply moves to the next bowler in a circular sequence using `(current_bowler + 1) % 6`.

```

1 void round_robin_scheduler() {
2     int prev = current_bowler;
3     current_bowler = (current_bowler + 1) % 6; // rotate to next
         bowler
4
5 }

```

OS Mapping: Each bowler is a process given one time quantum (one over = 6 balls). After the quantum expires, the CPU (pitch) is handed to the next process in a circular order.

Observation: Every bowler gets equal opportunity to bowl, and no single bowler dominates.

6.4 Priority Scheduling — Bowler Scheduling (Death Over)

For the final over (over 20, i.e., `match.score.overs == 19`), Round Robin is overridden and `priority_scheduler()` is called instead. This always assigns `current_bowler = 5`, which is the highest-skilled bowler (Bumrah / Boult with `skill = 45`).

```

1 void priority_scheduler() {
2     if (match.score.overs == 19) {
3         current_bowler = 5; // always pick highest priority
         bowler (index 5)
4     }

```

OS Mapping: Priority scheduling preempts the normal round robin rotation and forces the highest priority process onto the CPU. Here, the priority value is the bowler's `skill`

field, and the highest skill bowler (index 5, skill = 45) is always scheduled for the most critical time slot the last over.

Observation: This reflects a real match strategy where the best bowler is saved for the death over.

7 Assumptions

These are the assumptions we made during the design and implementation of the simulator

- **Probability Distribution for Ball Outcomes**

Each ball outcome is generated using a pseudo random value $r = \text{rand}() \% 100$. The outcomes are assigned in a sequential range based manner, where each event have a segment in the range.

- **Extras:**

- * No-ball: 1%
- * Wide: 3%

- **Wicket:**

- * Base probability: 5%
- * Adjusted by skill difference:

$$pw = 5 - \frac{(\text{bat skill} - \text{bowler skill})}{8}$$

- * Between 2% and 10%
- * Increased by 2% in death overs (overs ≥ 19)

- **Dot Ball (0 runs):**

- * Base: $20\% + \frac{\text{bowler skill}}{8}$
- * Reduced by 3% in final over
- * Between 10% and 40%

- **Single (1 run):**

- * Base: $30\% + \frac{\text{bat skill}}{20}$
- * Reduced by 5% in death overs

- **Double (2 runs):**

- * Base: $10\% + \frac{\text{bat skill}}{25}$

- **Triple (3 runs):**

- * Fixed probability: 2%

– **Boundaries:**

* Four:

$$p4 = 8 + \frac{\text{bat skill}}{6}$$

* Six:

$$p6 = 3 + \frac{\text{bat skill}}{10}$$

* Both increase in death overs:

- +5% for fours
- +4% for sixes

* Reduced for weak batsmen (`skill < 15`)

* Max:

- Four: 30%
- Six: 15%

- **Fixed Number of Bowlers and Death Over Rule**

We assume exactly 6 bowlers per team. Bowlers are rotated using Round Robin scheduling till 19 overs. The final over (20th over) is a **death over**, where a fixed high-skill (assumed to be Bumrah (India), Boult(NZ)) bowler is always selected using priority scheduling.

- **Wide and No-Ball Assumptions**

Both wides and no-balls add 1 extra run to the score and do not count as a legal delivery. However, for simplicity, **no boundary off a wide or no-ball is possible** they always result in exactly 1 extra run. The no ball sets `free_hit` flag = 1 (i.e. in next over the batsman cant get out due to bowled)

- **Fielder Activation is Probabilistic**

When the ball is hit for 1, 2, or 3 runs, all 11 fielder threads are woken up via `ball_hit_cond`. Each fielder have **2% probability**.

- **Only One Fielder Can Cause a Run-Out**

Even if multiple fielders activate on the same ball, only the **first one to acquire `crease_mutex`** is considered the winning fielder. The winning fielder's identity is recorded in `winning_fielder_id`.

- **Batsman Skill is Fixed (Non-Adaptive)**

Each batsman is assigned a fixed `job_length` value at the start of the match which does not change based on form, match situation, or pitch conditions.

- **Deadlock Requires Both Batsmen Waiting Simultaneously**

A deadlock run-out is only triggered when **both `striker_waiting` and `nonstriker_waiting`** flags are set to 1 at the same time. If only one batsman fails to acquire the opposite end, the run simply does not complete but no wicket falls.

- **Match is Simulated Twice (SJF then FCFS)**

The entire match both innings is simulated twice(ie. SJF and FCFS) in the same program run. The first simulation uses SJF for batsman scheduling and the second uses FCFS. The toss result and teams remain the same across both runs.

- **Umpire and Third Umpire Are Not Separate Threads**

These roles are handled directly by the **striker thread and the main thread** rather than as dedicated umpire threads. Decisions like run-out, deadlock resolution, and innings management are made inline within the existing thread logic.

- **Waiting Time, Turnaround Time, and Burst Time Definitions**

We assume that the **arrival time of every batsman is 0**, (i.e., all batsmen are considered to be present in the system from the beginning of the innings.)

- **Turnaround Time (TAT)** is defined as the total time from the start of the innings until the batsman is dismissed, measured as total number of balls played
- **Burst Time (BT)** is defined as the number of balls faced by the batsman .
- **Waiting Time (WT)** is calculated as:

$$WT = TAT - BT$$

This represents the time the batsman spends waiting before match ends or batsman gets out.

- **Middle Order Batsman**

We assume that the **middle order batsmen correspond to positions 3 to 6** in the batting lineup.

- **Balls corresponds to Time**

Time is measured in terms of **balls**, and all scheduling and performance metrics (waiting time, turnaround time, burst time) are measured using ball count rather than real clock time.

8 Conclusion

This project shows the practical application of the basic concepts of the Operating System, which can be used for simulating a real-world scenario like a T20 Cricket Match. The simulator treats players as threads and their interactions during a match as synchronized events, successfully demonstrates concurrency, shared resource access, and coordination among multiple processes. The basic concepts of multithreading, mutex-based synchronization, semaphores, condition variables, and scheduling algorithms were successfully implemented and studied. The simulator also successfully demonstrated real-world issues

like race conditions during run-outs and deadlocks due to circular wait, and their resolutions. This simulator provides a better understanding of process management and the overall behavior of a dynamic concurrent system.